

TITLE: Implement a Java program to demonstrate inheritance and interfaces using suitable real-world examples.

Problem Statement

Implement a Java program to demonstrate inheritance and interfaces using suitable real-world examples.

Learning Objectives

To implement inheritance and interfaces for achieving code reusability and abstraction.

Theory

Inheritance allows one class to acquire the properties and methods of another class, promoting code reuse and reducing redundancy. Java supports single inheritance through the extends keyword. Interfaces define a contract that implementing classes must follow, enabling abstraction and multiple inheritance of type. Together, inheritance and interfaces support polymorphism and modular software development. They are fundamental concepts of object-oriented programming.

Conclusion

This Java program effectively demonstrates the practical application of inheritance and interfaces to achieve code reusability and abstraction. Through class inheritance (extends), child classes successfully acquire common attributes and behaviors from base parent classes, eliminating redundant code. Concurrently, using interfaces (implements) establishes a strict operational contract, allowing disparate product and shape types to enforce standardized methods like tax calculation and area computation. Together, these core Object-Oriented Programming concepts enable compile-time and runtime polymorphism, producing modular, scalable, and maintainable software architecture.

Exercise

1. Create a **Shape application** where Circle and Rectangle inherit from Shape and calculate area.

Code:

PTO

```

1  abstract class Shape {
2      |   abstract double calculateArea();
3  }
4  class Circle extends Shape {
5      |   private double radius;
6      |   public Circle(double radius) {
7      |       |   this.radius = radius;
8      |       }
9      |   @Override
10     |   double calculateArea() {
11     |       |   return Math.PI * radius * radius;
12     |       }
13     }
14  class Rectangle extends Shape {
15      |   private double length;
16      |   private double width;
17      |   public Rectangle(double length, double width) {
18      |       |   this.length = length;
19      |       |   this.width = width;
20      |       }
21      |   @Override
22      |   double calculateArea() {
23      |       |   return length * width;
24      |       }
25      }
26  public class ShapeApp {
27      |   Run main | Debug main
28      |   public static void main(String[] args) {
29      |       |   Shape circle = new Circle(5.0);
30      |       |   Shape rectangle = new Rectangle(4.0, 6.0);
31      |       |
32      |       |   System.out.println("Circle Area: " + circle.calculateArea());
33      |       |   System.out.println("Rectangle Area: " + rectangle.calculateArea());
34      |       }
35  }

```

Output:

```

Listening on 56263
User program running
User program finished
Circle Area: 78.53981633974483
Rectangle Area: 24.0

```

2. Develop a **E-Commerce Product System** where electronic, clothing, and grocery products inherit common properties and implement product interfaces.
PTO

Code:

```
1 interface Taxable {
2     double calculateTax();
3 }
4 class Product {
5     String name;
6     double price;
7     Product(String name, double price) {
8         this.name = name;
9         this.price = price;
10    }
11 }
12 class ElectronicProduct extends Product implements Taxable {
13     ElectronicProduct(String name, double price) { super(name, price); }
14     public double calculateTax() { return price * 0.18; }
15 }
16 class ClothingProduct extends Product implements Taxable {
17     ClothingProduct(String name, double price) { super(name, price); }
18     public double calculateTax() { return price * 0.05; }
19 }
20 class GroceryProduct extends Product implements Taxable {
21     GroceryProduct(String name, double price) { super(name, price); }
22     public double calculateTax() { return 0.0; }
23 }
24 public class ECommerceApp {
25     Run main | Debug main
26     public static void main(String[] args) {
27         Taxable[] products = {
28             new ElectronicProduct("Laptop", 1000),
29             new ClothingProduct("Shirt", 40),
30             new GroceryProduct("Milk", 3)
31         };
32         for (Taxable p : products) {
33             Product prod = (Product) p;
34             System.out.println(prod.name + " - Tax: $" + p.calculateTax());
35         }
36     }
37 }
```

Output:

```
Listening on 55466
User program running
User program finished
Laptop - Tax: $180.0
Shirt - Tax: $2.0
Milk - Tax: $0.0
```